Coding Project Final Report

Wizard Duel

Group 22

Prepared by
Alan Chen, Ferdinand Lim, Kareem Muftee, Xavier Hale
CS 440
University of Illinois Chicago

Table of Contents

List of Figures

List of Tables

I Project Description

1 Project Overview

Wizard Duel is a videogame around a battle of spells and casts between two wizards. In this, the player wizard fights against an enemy wizard AI until either one gets knocked out of bounds when all health points are depleted. Both wizards can cast a multitude of spells ranging from attack, defense, arcing (artillery-based) and zone based. Each spell has a different cost to their respective wizard's energy that replenishes over time.

The concept, most mechanics, and UI is derived from the Magic Missile Wizard Duel Development Project Description Report by Group 3 from Spring 2024, CS 440. However, this development does include several deviations from the report's design that were made in consideration of a better overall game experience for the player, and after evaluation of time/work cost to effectiveness in production.

2 Project Domain

The game starts off in the main menu screen, with four button options for the player to choose from: Play, Spell Book, Host, and Join. In this first product demo, only Play is fully functional, as Spell Book would require more time and spell types to be a fully fledged spell selection function, and Host and Join require further time and resources such as dedicated servers to share matches between players on different devices.

Play would immediately start the game, where the player is able to move the wizard around and attempt to cast selected spells. The same goes for the enemy wizard that is controlled by an AI. Spell casts are dependent on energy levels of their respective wizard, and require a sufficient amount of energy to be stored up in order for a successful spell cast. These energy levels are represented by the Orange-Coloured Energy Bar below the wizard, and slowly fills over time. Different spell types have different energy costs.

Some spell casts require time to charge up, and this is reflected in the amount of momentum the attack gets when initially launched. These spells, such as the Charge Spell, are aimed by the user's mouse and will arc due to bullet drop (gravity), forming an 'artillery' mechanic. Other spells such as the Ground Attack Spell and Lightning Spell instead have an 'area-of-effect' aiming mechanic where a marker moves across the battlefield as the LMB is held down, and cast on said spot when released. Other defensive spells as the Wall Spell are static, and remain in place until either hit or after several seconds. These are aimed with the player's mouse and are solid objects (attacks will not phase through these walls).

Certain spells like the Charge Spell are physical objects, so they can also be destroyed by hitting them with a spell of the same physical type. Spells do damage when they directly collide with the wizards, and this is reflected by the green health bar. Only the heal spell can replenish this at the cost of some energy points. When either

wizard's health goes to 0, it is knocked out of the map and the other wins. A 'match-end' screen will appear showing who won with the options to either restart the match or return to the menu screen.

3 Relationship to Other Documents

The idea and concepts of this project originate from both the Magic Missile Wizard Duel: Final Project Summary and Final Project Report, by A. You, E. Mendoza, K. Cordero, and N. Sheri. However, the documentation and design goals were not entirely followed due to design decisions that would lower the quality of gameplay, are beyond the scope of a prototype stage, or need additional resources we do not have.

4 Naming Conventions and Definitions

Refer to documentation of the same section in Magic Missile Wizard Duel: Final Project Report and Table of Key Terms below.

4a Definitions of Key Terms

Terms	**Meaning/Referencing**
Game	The entire program as a whole, including menus, scripts, classes and their programs.
Match	The active match between the wizards only. Encompasses only gameplay assets, objects and mechanics.
Cast	The action of using a spell, both offensively and defensively.
Spell	The actual manifestation and mechanics of the attack/defense itself.
Objects	The assets that can be visibly seen by the player throughout the game. These can be physical (has a solid body) or UI (other objects can phase through them)
Mechanics/Mechanisms	The inner code of assets that determines how they interact with the objects around them.
UI	User Interface layer for the actual game scene itself. This includes health bars, charge bars and energy bars, Not referring to the menu or end-game

	scenes
LMB	Left Mouse Button, used to interact with the program.

Table 1 - Table of Key Terms

4b UML and Other Notation Used in This Document

This document does not include any UML diagrams or others requiring notation within this premise. For a UML description, please refer to Magic Missile Wizard Duel: Development Project Design Report.

4c Data Dictionary for Any Included Models

Data Structure	**Content**
WizardPlayerScript	maxHealth (float) currentHealth (float) moveSpeed (float) HealthBar (HealthBar) minX (float) maxX (float) isDead (bool)
WizardEnemyScript	maxHealth (float) currentHealth (float) HealthBar (HealthBar) EnemyAI(EnemyAIScript) isDead (bool)
PlayerEnergyScript	maxEnergy (float) currentEnergy (float) energyRegenPerSecond (float) EnergyBarFill (Image) maxEnergy (float)
EnemyEnergyScript	maxEnergy (float) currentEnergy (float) energyRegenPerSecond (float) EnergyBarFill (Image) maxEnergy (float)
PlayerChargeSpellScript	chargeBarContainer (GameObject) chargeBarFill (Image) maxChargeTime (float)

	maxLaunchForce (float) currentCharge (float) energyCost (float)
EnemyChargeSpellScript	chargeBarContainer (GameObject) chargeBarFill (Image) maxChargeTime (float) maxLaunchForce (float) currentCharge (float) energyCost (float)
WallDefenseSpellScript	wallPrefab (GameObject) castPoint (Transform) wallDuration (float) coolDown (float) nextCastTime (float)
EnemyWallDefenseScript	wallPrefab (GameObject) castPoint (Transform) wallDuration (float) coolDown (float) nextCastTime (float)
HotBarScript	chargeSpell (PlayerChargeSpellScript) HealSpell (HealSpell) wallDefenseSpellScript (wallSpell) slotCount (int) selectedSlotColor (Color) defaultColor (Color)

Table 2 - Table of Data Structures

II Project Deliverables

As of the current state of the game, the menu and match itself are in a complete state, with working UI, player and AI mechanics, and endgame scenarios. Several spells are in working order and used by the player and AI, including offensive, defensive, and area-of-effect spells.

The three screens in this release include the menu, match, and endgame screens that are run through in order, with the option to restart the match or return to the menu are the endgame screen. Both match and endgame screens include fully-functioning UI as buttons and state displays (observers) so the player knows the current state of things such as their energy levels and health. Only the 3 buttons Spellbook, Host, and Join are not in working order due to time and work constraints to fit within the short development period.

1 First Release

In the first release on February 27, 2026, the project was still undergoing development of its foundation and setup, so many assets and mechanisms were not yet developed. The existing system included the player & enemy wizard objects, the charge spell and shield spells, as well as among user interface components, player energy bar, player charge bar, and the enemy health bar. Only the player wizard launched spells as the enemy wizard had no AI programmed yet. None of the wizards had the ability to move around the map either. However, both wizards did interact with the spells by casting, taking damage, and getting knocked out of the map when their health dropped to 0. This was reflected in the existing UI at the time.

At the time, only the match scene existed, so there was no menu nor endgame scene that was displayed at the start and end of the game. When either wizard was knocked out, it falls off the map but nothing else happens and the game continues. There were also issues with the display scaling that would mess up the locations and sizes of each UI component across different platforms (Microsoft Vs Mac) which caused issues when sharing and merging the code on GitHub.

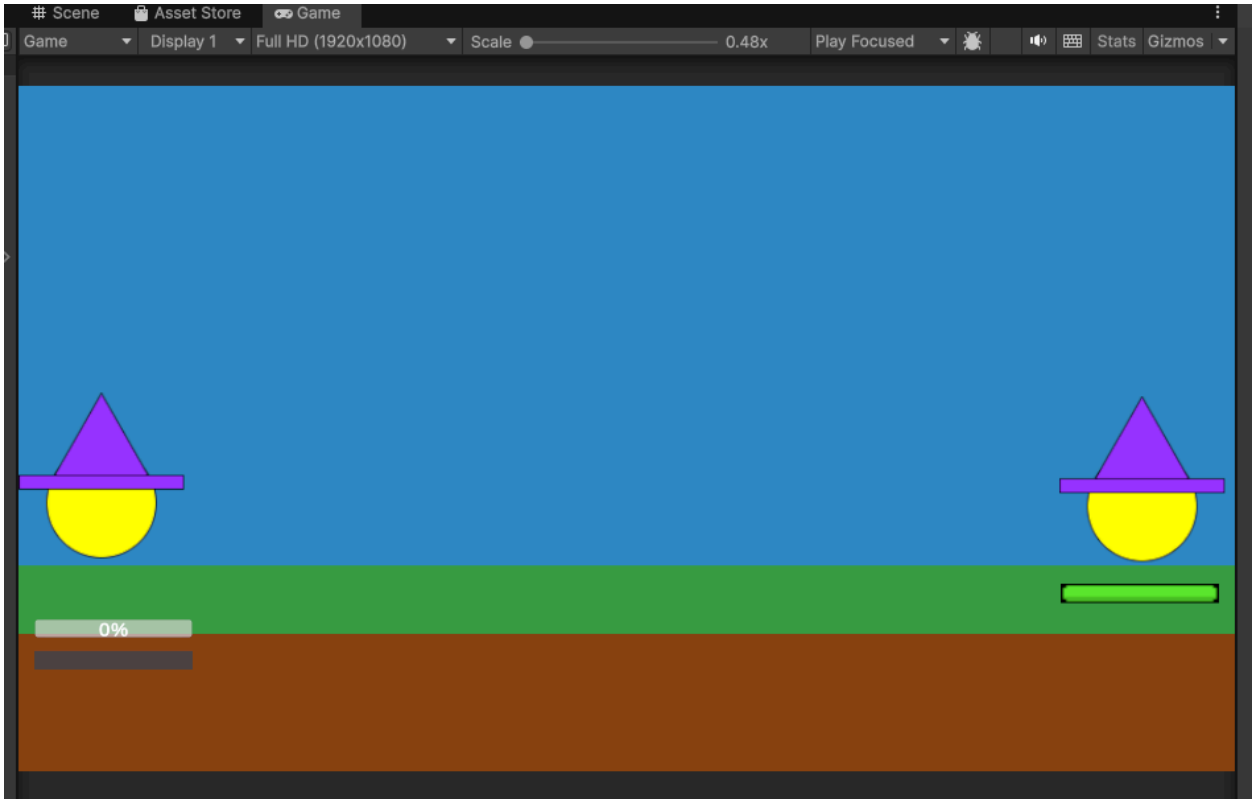

Figure 1 - Unfinished Game Scene

2 Second Release

In the second release on April 3, 2026, the game included the menu and endgame states, with the program starting off in the menu screen. The healthbar, chargebar, and

energy bars are fully visible and functional for both player and enemy wizards, and both could move around the map as they cast spells. Furthermore, the enemy wizard now has an AI that actively launches attacks, defends itself with the wall spell, and moves around at random. The AI actively tracks spells launched by the player wizard and will cast walls to defend itself in response.

In addition, the UI standardization bug was fixed from the previous release, further spells such as the Ground Attack Spell, Freeze Spell, and the spell switching mechanism were under development and still needed tweaks, meaning they did not make it to the demo but will be available in the final demo. In this final state, the game starts off in the game menu, and ends with the endgame state depending on whether the player wizard or enemy wizard triumphs over the other.

Figure 2 - Game Menu Scene

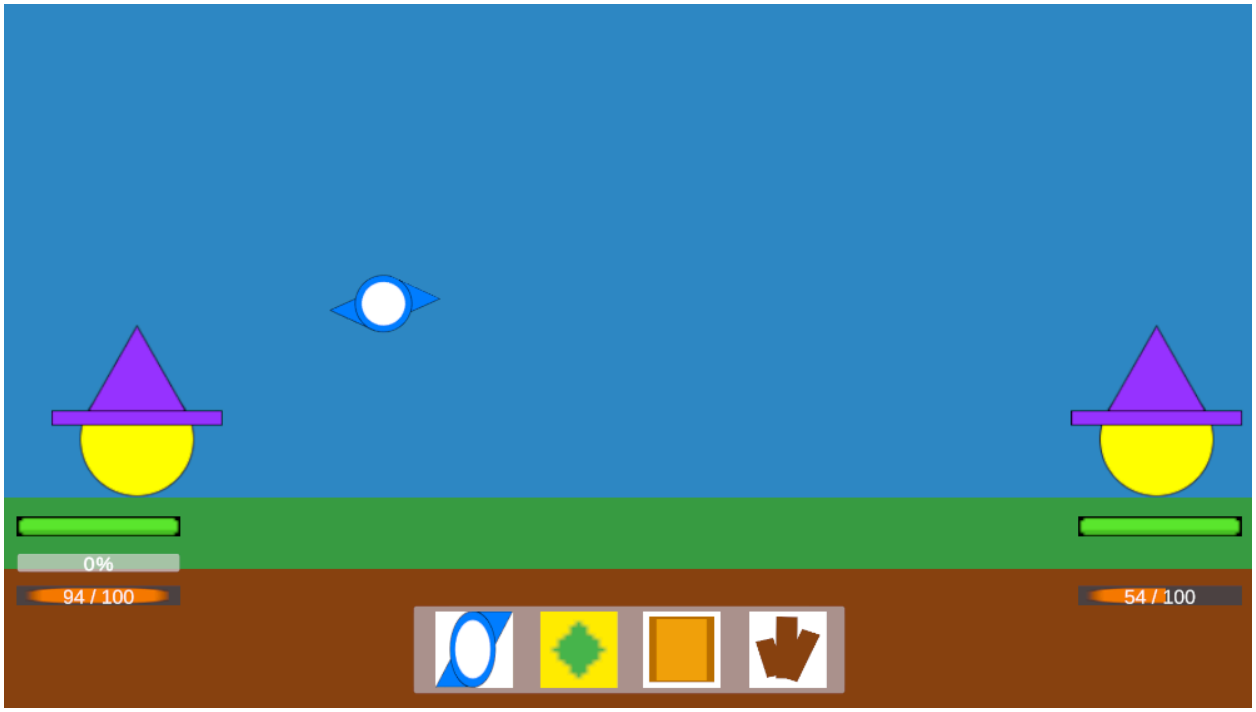

Figure 3 - Finished Prototype Game Scene

3 Comparison with Original Project Design Document

Although the foundation and design principles of this implementation follow closely to the original design document, one key difference was that the design document states that it is intended to be a multiplayer game that can match players on different devices through a server. This would require a network of servers to devices and the frameworks to support it. Given our small team and time constraints, we decided to opt for a single-player approach first as a prototype of the game, with the player fighting an AI that is run locally. This was much easier and allowed us to accomplish more in this short period of time.

There was also the decision to exclude the idea of Magic Missile Tracking, where the attack projectile would follow a path pre-drawn by the player. Although good on paper, this mechanic would almost guarantee a direct hit on the enemy each time, as the projectile behaves like a "cruise missile" that simply needs to fly directly straight to the enemy. This would not make for fun gameplay as it makes it too easy to win. Additionally, it would be easy for a player to use, but very hard to code an AI to match the same capability following this mechanic.

III Testing

1 Items to be Tested

- GS-01# - Game Scene Execution
- FP-01# - Fire Point

- CS-01# - Charge Spell
- CS-02# - Charge Spell UI
- WS-01# - Wall Spell
- EW-01# - Enemy Wizard AI Program
- HB-01# - Health Bar
- CSD-01# - Charge Spell Damage
- WD-01# - Wizard Death
- HS-01# - Heal Spell
- GAS-01# - Ground Attack Spell
- SS-01# - Spell Switching Mechanism
- WM-01# - Wizard Movement
- MM-01# - Main Menu
- PB-01# - Play Button
- VE-01# - Victory Endgame
- DE-01# - Defeat Endgame

2 Test Specifications

GS-01# - Game Scene Execution

Description: Execution of the game scene with the two wizards and ground runs without fail.

Items covered by this test: Ground, Player Wizard, Enemy Wizard

Requirements addressed by this test: F-6

Environmental needs: Unity Game Engine, Executable Scene

Intercase Dependencies: Not Applicable.

Test Procedures: Deploy and execute the scene, checking the terminal for any compiling and execution errors

Input Specification: N/A.

Output Specifications: Game scene appears with the two wizards, and zero errors in the terminal.

Pass/Fail Criteria: No errors pop up in the terminal on execution.

FP-01# - Fire Point Test

Description: All spells are launched from a “Fire Point” in front of the wizards

Items covered by this test: Player Wizard, Fire Point Object

Requirements addressed by this test: F-7

Environmental needs: Unity Game Engine, Executable Scene, Any Spell Object & Script.

Intercase Dependencies: GS-01

Test Procedures: Deploy and execute the scene, checking the terminal for any compiling and execution errors.

Input Specification: Left Mouse Button (LMB) click.

Output Specifications: When LMB clicked, a spell (any offensive spell) is cast from the fire point, and does not hit the wizard who cast it.

Pass/Fail Criteria: No terminal errors and the spell is cast correctly (depending on the type used)

CS-01# - Charge Spell Test

Description: An artillery type attack spell that launches a 'charge' projectile from the fire point towards where the mouse is. The initial momentum is dependent on the amount of charge upon release. When the projectile hits another object, it changes to its explosion effect .The energy level of the wizard that casts it should reduce by 20 points each time.

Items covered by this test: Fire Point Object, Charge Projectile, Explosion Render.

Requirements addressed by this test: F-8, DT-3, P-2

Environmental needs: Unity Game Engine, Executable Scene, Charge Spell Object Script.

Intercase Dependencies: GS-01, FP-01

Test Procedures: Deploy and execute the scene, then hold down the LMB to charge and aim with the mouse. Release to fire.

Input Specification: Left Mouse Button (LMB) click, hold and release.

Output Specifications: When LMB clicked, charging starts and continues as long as LMB is held. When LMB is released, it spawns the projectile at the fire point and flings it depending on the amount of charge and aim.

Pass/Fail Criteria: No terminal errors and the spell is cast correctly, spawning the charge projectile and flinging it. The projectile is also affected by bullet drop and explodes on contact with any object. Energy depletes by 20 points after each cast respectively.

CS-02# - Charge Spell UI Test

Description: The charge bar UI reflects the charging state of the charge spell when LMB is held.

Items covered by this test: Charge spell, Charge bar

Requirements addressed by this test: F-8, DT-3, P-2

Environmental needs: Unity Game Engine, Executable Scene, Charge Spell Object Script, Canvas UI.

Intercase Dependencies: CS-01, GS-01, FP-01

Test Procedures: Deploy and execute the scene, and initiate the charge spell by holding the LMB button.

Input Specification: Left Mouse Button (LMB) click and hold.

Output Specifications: When LMB clicked, a charge bar will appear below the wizard's side and fill up accordingly to when the LMB is held and released.

Pass/Fail Criteria: No terminal errors and the charge bar fits envisioned wireframe drafts, filling up from 0%.

WS-01# - Wall Spell Test

Description: Spawns a solid wall object in front of the casting wizard in the direction of where the mouse is pointing (for the player). Any attacks will collide with the wall (blocks them).

Items covered by this test: Wall Spell

Requirements addressed by this test: F-8, DT-3, P-2

Environmental needs: Unity Game Engine, Executable Scene, Wall Spell Object and Script

Intercase Dependencies: CS-01, GS-01, FP-01, EW-01 (For Enemy Wizard)

Test Procedures: Deploy and execute the scene, select the wall spell and cast wall.

Input Specification: Left Mouse Button (LMB) click.

Output Specifications: When LMB clicked, a wall spawns at the fire point facing where the mouse is aimed. Any attack spells are blocked when they collide with the wall spell.

Pass/Fail Criteria: No terminal errors and the wall spell works as described.

EW-01# - Enemy Wizard AI Program

Description: The Enemy Wizard is able to cast spells to attack and defend itself.

Items covered by this test: Enemy Wizard, Wizard AI, Attack Spell, Wall Spell

Requirements addressed by this test: Not Applicable.

Environmental needs: Unity Game Engine, Executable Scene, Charge Spell Object Script, Wall Spell Object Script, Wizard AI Script

Intercase Dependencies: CS-01, CS-02, GS-01, FP-01

Test Procedures: Deploy and execute the scene, and observe the enemy wizard for attack casts using the charge spell. Shoot charge spells at the enemy wizard and observe for wall spell casts in defense. Observe the enemy's charge bar and energy bars for proper function.

Input Specification: Left Mouse Button (LMB) click and hold for spells.

Output Specifications: Enemy wizard casts its own charge spells from its own fire point, and reacts with wall spells in response to the player wizard's charge spell sometimes.

Pass/Fail Criteria: No terminal errors, the charge spells launched by the enemy wizard are aimed at and around the player wizard (sometimes hit, sometimes miss), and the enemy wizard on occasion reacts with wall spell casts towards player-launched incoming charge spells. Energy buildup within the energy and charge bars fill and decrement as appropriate.

HB-01# - HealthBar Test

Description: A health bar for each wizard that reflects the amount of health denoted by the health variable in the wizard's scripts (max 100, min 0).

Items covered by this test: Enemy Wizard, Player Wizard.

Requirements addressed by this test: Not Applicable

Environmental needs: Unity Game Engine, Executable Scene, Player-Wizard Script, Enemy-Wizard Script

Intercase Dependencies: GS-01, FP-01

Test Procedures: Deploy and execute the scene, and observe a green health bar for each of the wizards. Hardcode levels in the Start() function to see health bar statuses (100, 50, 25, 0)

Input Specification: Not Applicable.

Output Specifications: Health bars appear on the UI layer.

Pass/Fail Criteria: No terminal errors, and health bars reflect the health variable of their respective health variable (e.g. when health of player wizard is 50, player's health bar should be half full and the enemy's health bar should be full.).

CSD-01# - Charge Spell Damage Test

Description: When a charge projectile hits either wizard, applies damage to the wizard's health variable and reflects change in health in the health bar.

Items covered by this test: Player Wizard, Enemy Wizard, Wizard AI, Charge Attack Spell, Health Bar UI.

Requirements addressed by this test: Not Applicable

Environmental needs: Unity Game Engine, Executable Scene, Charge Spell Object Script, Player-Wizard Script, Enemy-Wizard Script, Wizard AI Script, Canvas UI

Intercase Dependencies: CS-01, GS-01, FP-01, HB-01

Test Procedures: Deploy and execute the scene, and launch a charge spell that collides with the enemy wizard. Observe for health bar decrements. Conduct the same for the player wizard.

Input Specification: Left Mouse Button (LMB) click and hold for spells.

Output Specifications: When the charge spell collides with a wizard, their respective health variable decrements and is reflected in their healthbar.

Pass/Fail Criteria: No terminal errors, the charge spells launched without any terminal or spawn errors, and the health and healthbar decrements by 25 points.

WD-01# - Wizard Death Test

Description: When the health reaches 0, the wizard will enter a dead state where it cannot cast, and is knocked out of the battle.

Items covered by this test: Enemy Wizard, Player Wizard.

Requirements addressed by this test: Not Applicable

Environmental needs: Unity Game Engine, Executable Scene, Player-Wizard Script, Enemy-Wizard Script

Intercase Dependencies: GS-01, EW-01

Test Procedures: Deploy and execute the scene, and damage the player wizard until it dies (by enemy attack or self-infliction) to test player wizard. Kill the enemy wizard next execution to test it.

Input Specification: LMB to cast spells.

Output Specifications: Respective wizard dies, is kicked out of the scene and cannot cast anymore.

Pass/Fail Criteria: No terminal errors, and wizards die as described.

HS-01# - Heal Spell Test

Description: A healing spell that consumes 30 energy points to restore 25 points of health. Does not exceed the capacity of 100 health points if overflow. Change in health points is reflected by the health bar.

Items covered by this test: Enemy Wizard, Player Wizard, Health Bars

Requirements addressed by this test: Not Applicable

Environmental needs: Unity Game Engine, Executable Scene, Player-Wizard Script, Enemy-Wizard Script, Health Script

Intercase Dependencies: GS-01, HB-01

Test Procedures: Deploy and execute the scene, and take damage (either through waiting for enemy attack or launching attack at yourself). Then select the heal spell and cast.

Input Specification: LMB click.

Output Specifications: Health bar regains points after cast as it displays the state of health.

Pass/Fail Criteria: No terminal errors, and health is regained as described.

GAS-01# - Ground Attack Spell Test

Description: A spell that shoots out a brown aim point from a fire point that travels underground as long as the user holds down the LMB. When released, spikes erupt from the ground where the aim point is. If these spikes collide with a wizard, then damage is dealt.

Items covered by this test: Enemy Wizard, Player Wizard, Ground Attack Spell

Requirements addressed by this test: Not Applicable

Environmental needs: Unity Game Engine, Executable Scene, Player-Wizard Script, Enemy-Wizard Script, Health Script

Intercase Dependencies: GS-01, HB-01

Test Procedures: Deploy and execute the scene, and take damage (either through waiting for enemy attack or launching attack at yourself). Then select the heal spell and cast.

Input Specification: LMB click.

Output Specifications: Health bar regains points after cast as it displays the state of health.

Pass/Fail Criteria: No terminal errors, and health is regained as described.

SS-01# - Spell Switching Mechanism Test

Description: Spells are tied to keys 1, 2, 3, and 4 to switch between spells (1 - charge spell, 2 - heal spell, 3 - wall spell, 4 - ground attack spell). Spells can be quickly selected by pressing these keys. Spell selection and selected spell are reflected by a UI on screen to show options and selected.

Items covered by this test: Player Wizard, Charge Spell, Heal Spell, Wall Spell, Ground Attack Spell, Canvas UI

Requirements addressed by this test: Not Applicable

Environmental needs: Unity Game Engine, Executable Scene, Player-Wizard Script, Charge Spell Script, Heal Spell Script, Wall Spell Script, Ground Attack Spell Script

Intercase Dependencies: GS-01, CS-01, HS-01, WS-01, GAS-01

Test Procedures: Deploy and execute the scene, and press the number keys to switch spells. See that this change is reflected in the spell cast when LMB is pressed, and in the spell selection UI on screen.

Input Specification: LMB click and hold, 1 2 3 and 4 keys.

Output Specifications: Spell cast and UI selection should reflect the spell selected.

Pass/Fail Criteria: No terminal errors, and spell switching works described.

WM-01# - Wizard Movement Test

Description: Both wizards should be able to move on their respective sides. They cannot go out of the scene, nor to each other's sides. A to move left and D to move right for the player wizard. Their fire points should move along with them.

Items covered by this test: Enemy Wizard, Player Wizard,

Requirements addressed by this test: Not Applicable

Environmental needs: Unity Game Engine, Executable Scene, Player-Wizard Script, Enemy-Wizard Script, Enemy Wizard AI, Fire Point

Intercase Dependencies: GS-01, FP-01, EW-01

Test Procedures: Deploy and execute the scene. For the player wizard, use A and D keys to move. For the enemy wizard, wait for its AI to decide to move to another spot.

Input Specification: A and D keys.

Output Specifications: Wizards can move left and right in their respective zones.

Pass/Fail Criteria: No terminal errors, movement is smooth and works as described.

MM-01 - Main Menu Test

Description: The game should start from the main menu and the play button should be clickable.

Items covered by this test: Main Menu manager, play button

Requirements addressed by this test: Not Applicable

Environmental needs: Unity Game Engine, Main menu scene

Intercase Dependencies: Not Applicable.

Test Procedures: Upon starting the game, the main menu should appear first. When the play button is pressed, there should be feedback indicating it was pressed.

Input Specification: LMB click

Output Specifications: The user should be able to

Pass/Fail Criteria: There are no errors and the game starts at the main menu as intended. The Play button is clickable and offers feedback to the user indicating it was pressed.

PB-01 - Play Button Test

Description: The user should be able to immediately jump into a match with an enemy NPC using the Play button in the main menu.

Items covered by this test: Sample Scene manager, Play Button

Requirements addressed by this test: F-4, F-6

Environmental needs: Unity Game Engine, Main Menu Scene, SampleScene, MainMenu Script

Intercase Dependencies: MM-01

Test Procedures: After executing the scene, the game's main menu should be displayed along with all four of the main menu buttons. Clicking the Play button should send the user to a match with the AI NPC.

Input Specification: LMB click

Output Specifications: The scene changes to the game scene and allows the user to play a match with the NPC

Pass/Fail Criteria: The scene changes from the main menu to the game scene upon clicking the Play button as intended. The user is able to play a match against the NPC.

VE-01 - Victory Endgame Test

Description: Upon winning the game against the enemy AI, an endgame scene should be displayed and should let the user know they have won and allow them to either return to the main menu or play another match.

Items covered by this test: Endgame Scene manager, play button, main menu button

Requirements addressed by this test: Not Applicable.

Environmental needs: Unity Game Engine, Game scene, Endgame scene, Main menu scene, Endgame manager script

Intercase Dependencies: GS-01, MM-01

Test Procedures: Win 2 matches against the enemy AI and check if an endgame scene appears indicating the user has won. After each match press either the play again button or main menu button and check if both buttons work as intended.

Input Specification: LMB click

Output Specifications: The victory endgame scene must be displayed along with both main menu and play again buttons and both buttons should perform their respective functions.

Pass/Fail Criteria: Upon winning the match, the victory endgame scene successfully appears and both the play again and main menu buttons work as intended.

DE-01 - Defeat Endgame Test

Description: Upon losing the game against the enemy AI, an endgame scene should be displayed and should let the user know they have lost the match and allow them to either return to the main menu or play another match

Items covered by this test: Endgame Defeat Scene manager, play button, main menu button

Requirements addressed by this test: Not Applicable.

Environmental needs: Unity Game Engine, Game scene, Endgame Defeat scene, Main menu scene, Endgame manager

Intercase Dependencies: GS-01, MM-01

Test Procedures: Lose 2 matches against the enemy AI and check if an endgame scene appears indicating the user has won. After each match, press either the play again button or main menu button and check if both buttons work as intended.

Input Specification: LMB click

Output Specifications: The defeat endgame scene must be displayed along with both main menu and play again buttons and both buttons should perform their respective functions.

Pass/Fail Criteria: Upon losing the match, the defeat endgame scene successfully appears and both the play again and main menu buttons work as intended.

3 Test Results

GS-01# - Game Scene Execution Testing

Date(s) of Execution: 02/16/2026

Staff conducting tests: Alan Chen, Ferdinand Lim

Expected Results: Game scene is compiled and executed without any terminal errors.

Actual Results: No errors, game scene starts as expected.

Test Status: Pass

FP-01# - Fire Point Testing

Date(s) of Execution: 02/20/2026

Staff conducting tests: Ferdinand Lim, Kareem Muftee

Expected Results: Spells are launched from the exact position that fire point is located on the editing screen, and does not collide with the wizard firing it

Actual Results: Works as described and moves with the wizard.

Test Status: Pass

CS-01# - Charge Spell Testing

Date(s) of Execution: 02/21/2026

Staff conducting tests: Ferdinand Lim, Xavier Hale

Expected Results: When selected, holding down the LMB charges up power that is reflected by the charge bar. After release, built up energy affects the initial momentum of the charge projectile and aiming is done by the mouse. Bullet drop of the projectile is expected.

Actual Results: Works as described, no issues.

Test Status: Pass

CS-02# - Charge Bar UI Testing

Date(s) of Execution: 02/21/2026

Staff conducting tests: Ferdinand Lim, Xavier Hale

Expected Results: When the charge spell is selected and the LMB is held down, a charge bar appears, starting at 0% and climbs as long as LMB is held. When released, the charge bar disappears and its energy status is reflected in the momentum of the charge projectile. Works for both player and enemy wizards. Energy depletes by 20 points after each cast respectively.

Actual Results: Works as described for both player and enemy wizards with their own respective charge bars.

Test Status: Pass

WS-01# - Wall Spell Testing

Date(s) of Execution: 02/23/2026, 03/17/2026

Staff conducting tests: Kareem Muftee, Ferdinand Lim

Expected Results: When selected and LMB is clicked, a wall spawns in front of the wizard at fire point's distance, and faces wherever the mouse points. Any attack spell like charge spell that collides with the wall is blocked.

Actual Results: Works as described for both player and enemy wizards. Tested first for player wizard alone and a second time after for the enemy wizard once Enemy Wizard AI was completed

Test Status: Pass

EW-01# - Enemy Wizard AI Testing

Date(s) of Execution: 03/17/2026

Staff conducting tests: Ferdinand Lim, Xavier Hale

Expected Results: The enemy wizard is able to fire charge spells at the player wizard with a randomized accuracy (sometimes miss, sometimes hit) and charge build up. The enemy wizard is also able to deploy wall spells in reaction to enemy attack casts at times along projected trajectories.

Actual Results: Works as described and enemy wizard's actions are randomized to prevent stale gameplay

Test Status: Pass

HB-01# - Health Bar Testing

Date(s) of Execution: 02/25/2026

Staff conducting tests: Alan Chen, Xavier Hale

Expected Results: Health bars are clear and tied to health variables of each wizard respectively. Damage taken and heals are reflected by the state of the health bar.

Actual Results: Works as described and the function to add or subtract health points for damage is easily callable by other scripts.

Test Status: Pass

CSD-01# - Charge Spell Damage Testing

Date(s) of Execution: 02/25/2026

Staff conducting tests: Ferdinand Lim, Kareem Muftee

Expected Results: When a charge spell projectile collides with a wizard, does 25 points of damage to the respective wizard's health, and is reflected on its health bar.

Actual Results: Works as described, and both wizards are able to deal damage to the other or themselves (by accident).

Test Status: Pass

WD-01# - Wizard Death Testing

Date(s) of Execution: 02/25/2026

Staff conducting tests: Ferdinand Lim, Kareeem Muftee

Expected Results: When either wizard's health falls to 0, the wizard is knocked out and cannot cast any more spells.

Actual Results: Works as described, and the killed wizards enters its dead state as expected

Test Status: Pass

HS-01# - Heal Spell Testing

Date(s) of Execution: 02/26/2026

Staff conducting tests: Xavier Hale, Alan Chen

Expected Results: When a cast consumes 30 energy points and regains 25 health points. Change in health is reflected by the health bar. Does not overflow 100.

Actual Results: Works as described.

Test Status: Pass

GAS-01# - Ground Attack Spell Testing

Date(s) of Execution: 04/06/2026

Staff conducting tests: Kareem Muftee, Alan Chen

Expected Results: When cast the brown aim point travels across the screen as long as LMB is held, and when released, spikes erupt that damage the wizards if touched. Spikes disappear after a second.

Actual Results: Works as described, no issues with wizards physically colliding with spikes.

Test Status: Pass

SS-01# - Spell Switching Mechanism Testing

Date(s) of Execution: 04/10/2026

Staff conducting tests: Alan Chen, Ferdinand Lim

Expected Results: When cast the brown aim point travels across the screen as long as LMB is held, and when released, spikes erupt that damage the wizards if touched. Spikes disappear after a second.

Actual Results: Works as described, no issues with wizards physically colliding with spikes.

Test Status: Pass

WM-01# - Wizard Movement Testing

Date(s) of Execution: 03/26/2026

Staff conducting tests: Ferdinand Lim, Xavier Hale

Expected Results: Player wizard can move with A and D keys, and enemy wizard can move based on its AI. Both cannot move out of the scene nor cross onto each other's side.

Actual Results: Works as described, no issues with wizards moving and hitting boundaries (they stop). Wizards movements are smooth.

Test Status: Pass

MM-01# - Main Menu Testing

Date(s) of Execution: 03/09/2026

Staff conducting tests: Xavier Hale, Ferdinand Lim

Expected Results: The execution of the game starts at the main menu, with a clickable play button.

Actual Results: Works as described, no issues with the formatting and layout of the main menu scene.

Test Status: Pass

PB-01# - Play Button Testing

Date(s) of Execution: 03/09/2026

Staff conducting tests: Xavier Hale, Ferdinand Lim

Expected Results: When the play button in the main menu is clicked, immediately switches to a new game scene with the player and enemy wizards.

Actual Results: Works as described, no issues scene switch and execution of the game scene

Test Status: Pass

VE-01# - Victory Endgame Testing

Date(s) of Execution: 03/11/2026

Staff conducting tests: Xavier Hale, Ferdinand Lim

Expected Results: When the player defeats the enemy wizard, an endgame scene with a message to let the player know they won is displayed. Buttons to the main menu or start another match work accordingly.

Actual Results: Works as described, no issues with scene switch, victory message and buttons to different scenes.

Test Status: Pass

DE-01# - Defeat Endgame Testing

Date(s) of Execution: 03/12/2026

Staff conducting tests: Xavier Hale, Ferdinand Lim

Expected Results: When the enemy wizard defeats the player, an endgame scene with a message to let the player know they lost is displayed. Buttons to the main menu or start another match work accordingly.

Actual Results: Works as described, no issues with scene switch, defeat message and buttons to different scenes.

Test Status: Pass

4 Regression Testing

All tests are repeated informally whenever execution is run to test a new addition to the program. For example, to test a new spell, Game Execution, Fire Point, Health Bar, etc. are all tested in order to engage the new test since it will not work if any of the former layers do not.

IV Inspection

1 Items to be Inspected

1. **WizardPlayerScript.cs** : contributed by Ferdinand. Handles player health, damage, physics, and scene transition.
2. **EnemyAIScript.cs** : contributed by Xavier. Controls the enemy wizard's AI, movement, and spell casting.
3. **PlayerChargeSpellScript.cs** : contributed by Kareem. Handles charge buildup, projectile launch direction, and energy cost.
4. **HotBarScript.cs** : contributed by Alan Chen. Manages the spell hotbar UI and maps number key inputs to the active spell.

2 Inspection Procedures

Each team member independently reviewed the three scripts submitted by their teammates before a group meeting. Inspectors evaluated each script for the following:

- **Correct:** Does the code produce the behavior described in the test specifications?
- **Formatting:** Are variable and function names clear, and is the logic easy to follow without explanation?
- **Edge Cases:** Are edge cases handled, such as health reaching zero, energy being insufficient, or actions being attempted on a dead wizard?

If someone found a bug we discuss it during our group meetings, unless it was a small bug that wasn't worth the time. No external checklists were used. All identified issues were minor and were resolved in code after meetings.

3 Inspection Results

WizardPlayerScript.cs Inspected by: Xavier, Alan, and Kareem. 04/14/2026 Findings: A missing null check on the healthBar reference was also noted as a minor risk. Resolution: Null check added to the healthBar reference. Re-inspection: None required.

EnemyAIScript.cs Inspected by: Ferdinand, Alan, Kareem. 04/14/2026 Findings: The AI had no check for the frozen state, allowing it to continue acting while frozen. Action probability thresholds were uncommented magic numbers. Resolution: A frozen state check was added to Update() so all AI behavior halts while frozen. Comments were added to the probability thresholds. Re-inspection: None required.

PlayerChargeSpellScript.cs Inspected by: Ferdinand, Xavier, Alan. 04/14/2026 Findings: Core logic was correct. One edge case found: the charge bar UI was not hidden if the player wizard died mid-charge. Resolution: A dead-state check was added to hide the charge bar on wizard death. Re-inspection: None required.

HotBarScript.cs Inspected by: Ferdinand, Xavier, Kareem. 04/14/2026 Findings: Code was clean and readable with no logic errors. Resolution: None required.

V Recommendations and Conclusions

The primary recommendation for future development is the implementation of the Spellbook selection screen, multiplayer networking with the Host and Join buttons, and expanded enemy AI complexity. The project is considered complete and ready for final submission in its current state. Other than that all items covered in testing and inspection passed their criteria.

VI Project Issues

1 Open Issues

- **Spellbook Selection**

  The Spellbook selection in the menu is supposed to be a scene where the player can scroll through a list of all spells available and select 4 in a loadout to bring to battle. This menu should list the icon of each spell, as well as statistics of each such as energy cost, cast type (artillery / direct / etc), and attribute type (offensive / defensive / buff). From here the player can mix and match loadouts, and each game, any enemy AI would also pick a random loadout from the list of spells to use.

- **Join**

  With the initial design document detailing that this would be a multiplayer game, one of the components would be the ability to join games hosted by other players in waiting. This would connect both players on a server to play the match. However due to skill, manpower and time constraints, this was not a goal in the prototype stage.

- **Host**

  Another part that was present in the design document but was never targeted for the same reasons as Join. Players will have the option to host a game that can be joined by other players who click 'Join'. There can also be a list of players hosting for those that want to join their friends in a match.

2 Waiting Room

- **Tech-Tree System For Spells As Game Progression**

  There is potential for a tech-tree system that allows players to experience game progression outside of the current arcade-style the game currently has. This would mean points are collected after each battle and can be spent on unlocking new spells or upgrading current ones. Spells may even be organized in a tree system that would require certain prerequisites to be unlocked before learning this new spell with points. This would make for a more interesting game that players can come back to over time. This would require a sufficient amount of spells to be ready (about 25) and require extensive rework into the endgame scenes and spell book.

3 Ideas for Solutions

Unity would allow the spell's data (energy cost, damage, reach) to be stored and displayed. In a UI scrollable where the player clicks on spells to view them and their descriptions.

For multiplayer: Unit's netcode for GameObjects library would be more practical to start off with. Later on we could scale to better customized infrastructure. But Netcode is designed for small-scale matches without requiring dedicated servers.

For tech-tree progression, Unity's PlayerPrefs system could help be a small-scale solution to store unlocked spells and accumulated points between sessions.

4 Project Retrospective

What Went Well:

- **Separation of Tasks**

Separating tasks amongst ourselves went well as most components were only dependent on what we finished in earlier sprints. Assets such as projectile objects and UI could be done separately at different times amongst ourselves.

- **Usage of Unity Game Engine for this Project**

  Unity granted us an inbuilt foundation to create our objects and how they react to each other, including a physics engine for our spells. This allowed us to focus more on the aspects and objects within the game itself rather than costing us a lot of time working out the mechanics of the game.

What Did Not Go Well:

- **Merge Issues With Github Desktop**

  At times when working with Github Desktop, one of our members would get merge conflicts that could not be resolved locally, as the system did not allow him to commit any merges of his branch at all. We had to resort to having to import his files to another member's computer and reintegrate it into the system there, costing us a lot of time.

- **Wait Issues Due to Merge Conflicts**

  As a result of the merge issues with one of our members mentioned above, some components such as scripts to allow the enemy wizard to use the Ground Attack Spell were unable to be completed by the demo due to the delays and not having the dependent code present across the main branch.

VII Glossary

Bullet Drop: The downward arc of a projectile over time caused by gravity. Only applies to the Charge Spell.

Charge: The buildup of the Charge Spell. Holding the left mouse button increases charge level, which determines launch force on release.

Energy Bar: The orange UI bar below each wizard indicating current energy. Regenerates over time and is spent on spell casts.

Fire Point: A child object attached to each wizard marking the spawn position of all spell projectiles.

Health Bar: The green UI bar showing each wizard's current health out of a 100.

Hotbar: The on-screen UI showing the player's four selectable spells with the active spell highlighted.

NPC: Non-Player Character. The enemy wizard, controlled by EnemyAIScript rather than a human player.

Prefab: A reusable Unity GameObject template. Spell projectiles and UI elements are built as prefabs and used during gameplay.

Scene: A Unity term for a self-contained environment. This game uses three: Main Menu, Game (SampleScene), and Endgame.

VIII References / Bibliography

[1] A. You, E. Mendoza, K. Cordero, N. Sheri, Magic Missile Wizard Duel: Development Project Description Report, University Of Illinois At Chicago: CS 440, Spring 2024

[2] A. You, E. Mendoza, K. Cordero, N. Sheri, Magic Missile Wizard Duel: Project Final Report Summary, University Of Illinois At Chicago: CS 440, Spring 2024

[3] Unity Technologies, Unity User Manual 2022 [online]. Available: docs.unity3d.com

IX Index